BIRZEIT UNIVERSITY

Electrical and Computer Engineering

ENCS2380 – Computer Organization and Microprocessors- Spring 2025

Course Project: **Single Cycle Processor Design**

Project Objectives

- Designing a 32-bit processor with 24-bit instructions
- Using the Logisim simulator to model and test the single cycle processor
- Teamwork

Instruction Set Architecture

Here, you are required to design a RISC processor that has sixteen 32-bit registers (i.e. **R0** to **R15**). Register **R0** is hardwired/connected to zero, where reading **R0** always returns zero value and writing **R0** has no effect. The value written to **R0** is neglected.

All instructions are **24-bit** long and aligned in memory. **PC** register (20-bit wide) contains the instruction address. There are **three** instruction types, **R-type**, **I-type**, and **J-type** as shown below:

R-type

8-bit opcode (Op) and 4-bit register numbers (x, y, and z)

8-opcode	4-x	4-y	4-z	4-Not-Used

I-type

8-bit opcode (Op), 4-bit register numbers (x and y), and 8-bit Immediate

8-opcode	4-x	4-y	Imm8

J-type

8-bit opcode (Op) and 16-bit Immediate

8-opcode	Imm16

Register Use:

Register	**Number**	**Definition**	**Function**
Register x (**Rx**)	Destination register	**Rx** is the name and value of register **x**	Always written and never read
Register y (**Ry**)	First source register	**Ry** is the name and value of register **y**	Always read and never written
Register z (**Rz**)	Second source register for the R-type only	**Rz** is the name and value of register **z**	Always read and never written

Instruction Encoding:

R-type, I-type, and J-type instructions, definitions, and encodings are shown below:

Instruction	Definitions	Encoding			
AND	Rx = Ry & Rz	Op = 0	x	y	z
CAND	Rx = ~Ry & Rz	Op = 1	x	y	z
OR	Rx = Ry | Rz	Op = 2	x	y	z
XOR	Rx = Ry ^ Rz	Op = 3	x	y	z
ADD	Rx = Ry + Rz	Op = 4	x	y	z
NADD	Rx = –Ry + Rz	Op = 5	x	y	z
SEQ	Rx = (Ry == Rz) (result is 0 or 1)	Op = 6	x	y	z
SLT	Rx = (Ry < Rz) (result is 0 or 1)	Op = 7	x	y	z
ANDI	Rx = Ry & Imm8	Op = 8	x	y	Imm8
CANDI	Rx = ~Ry & Imm8	Op = 9	x	y	Imm8
ORI	Rx = Ry | Imm8	Op = 10	x	y	Imm8
XORI	Rx = Ry ^ Imm8	Op = 11	x	y	Imm8
ADDI	Rx = Ry + Imm8	Op = 12	x	y	Imm8
NADDI	Rx = –Ry + Imm8	Op = 13	x	y	Imm8
SEQI	Rx = (Ry == Imm8) (result is 0 or 1)	Op = 14	x	y	Imm8
SLTI	Rx = (Ry < Imm8) (result is 0 or 1)	Op = 15	x	y	Imm8
SLL	Rx = Shift_Left_Logical(Ry, Imm8)	Op = 16	x	y	Imm8
SRL	Rx = Shift_Right_Logical(Ry, Imm8)	Op = 17	x	y	Imm8
SRA	Rx = Shift_Right_Arithmetic(Ry, Imm8)	Op = 18	x	y	Imm8
ROR	Rx = Rotate_Right(Ry, Imm8)	Op = 19	x	y	Imm8
BEQ	Branch if (Rx == Ry) (PC += Imm8<<1)	Op = 20	x	y	Imm8
BNE	Branch if (Rx != Ry) (PC += Imm8<<1)	Op = 21	x	y	Imm8
BLT	Branch if (Rx < Ry) (PC += Imm8<<1)	Op = 22	x	y	Imm8
BGE	Branch if (Rx >= Ry) (PC += Imm8<<1)	Op = 23	x	y	Imm8
LW	Rx ← 4byte MEM[Ry + Imm8]	Op = 24	x	y	Imm8
SW	MEM[Ry + Imm8] ← 4byte Rz	Op = 25	z	y	Imm8
J	PC = PC + signed(Imm16<<1)	Op = 26	Imm16		
JAL	R15 = PC + 3; PC=PC+signed(Imm16<<1)	Op = 27	Imm16		

Instruction Description:

Opcodes 0 to 7: **R-type** ALU instructions and there are in total 8 instructions.

Opcodes 8 to 15: **I-type** ALU instructions. The immediate value considers as the second ALU operand and register **x** is the destination operand.

NOTES:

I-type ALU instructions from **ANDI** to **SLTI**: have same functionality of **R-type** instructions from **AND** to **SLT**, the destination register is **Rx** (not **Rz**) and the second operand is an immediate.

I-type encodes a 8-bit signed immediate **Imm8** with range -128 to +127.

Opcodes 16 to 19: four shift and rotate instructions (**SLL** to **ROR**). Shift or rotate value is the **Imm5** with values 0 to 31.

Opcodes 20 to 23: four branch instructions (**BEQ** to **BGE**) and PC-relative addressing mode. It updates the PC value as follows:

Branch is taken then **PC = PC + Imm8<<1**. Else, **PC = PC + 3**.

Opcodes 24 and 25: load word (**LW**) and store word (**SW**) instructions and these two instructions address 4-byte words in memory. For this case, displacement addressing is used and the effective memory address is = **Ry + Imm8**. Register **x** is a source for **SW** and a destination register for **LW**.

NOTE:

Loading and storing one byte or a half word is not defined to simplify the design.

Opcodes 26 and 27: Jump (**J**) and Jump-and-Link (**JAL**) instructions. These instructions are PC-relative addressing and to compute the jump target address:

PC = PC + sign_extend(Imm16<<1).

Also, **JAL** instruction stores the return address in **R15** (**R15 = PC + 3**).

Programming Notes:

NADD	**R1, R2, R3**	computes R1 = R3 – R2	(Rx = R1, Ry = R2, and Rz = R3)
SUB	**R1, R2, R3**	is a pseudo-instruction ➔	**NADD R1, R3, R2**
ADDI	**R1, R2, –1**	computes R1 = R2 – 1	(Rx = R1, Ry = R2, and Imm8 = –1)
NADDI	**R1, R2, 1**	computes R1 = 1 – R2	(Rx = R1, Ry = R2, and Imm8 = 1)
LI	**R1, 5**	is a pseudo-instruction ➔	**ORI R1, R0, 5**

PC Register and Instruction Memory:

Program counter (PC) is a special-purpose 20-bit register. It can address an **instruction memory** having 2^{20} bytes. All instruction addresses are multiple of 3 bytes.

Data Memory:

LW and SW instructions address a separate **data memory**. The data memory will be also restricted to 2^{20} bytes. Although the architecture is 32 bits, the size of the instruction and data memories will be restricted to 2^{20} bytes. This is because the **Logisim** tool supports only small memories.

Addressing Modes:

PC-relative addressing mode is used by all branch and jump instructions.

For taken branches: **PC = PC + sign_extend(Imm8<<1)**

For jumps (**J** and **JAL**): **PC = PC + sign_extend(Imm16<<1)**, where **Imm16** is only 16 bits.

For **LW** and **SW**, displacement addressing is used: **Address = Ry + sign_extend(Imm8)**

Register File:

Develop a Register file comprising sixteen 32-bit registers labeled from **R1** to **R15**, serving one write port and two read ports. **R0** is fixed at 0 and remains immutable. **R0** always reads as 0 and cannot undergo any write operations.

Arithmetic and Logic Unit (ALU):

Develop a 32-bit ALU to handle all the required operations:

XOR, AND, OR, CAND, ADD, SEQ, NADD, SLT, SRA, SRL, SLL, and ROR.

Program Execution:

The program will be loaded and will start at address 0 in the instruction memory. The data segment will be loaded and will start also at address 0 in the data memory. To terminate the execution of a program, the last instruction in the program can jump to itself indefinitely because there is no underlying operating system to terminate the program.

Develop a Single-Cycle Processor:

Develop the control and datapath components for the single-cycle processor and verify their correctness. You should have enough test cases that check ALL instructions mentioned in the instruction set. You should also write full codes (as requested in the Testing and Verification part) that demonstrate the execution of complete programs. To ensure the correctness of the design, display the contents of ALL registers from **R1** to **R15** clearly in your design's top-level (i.e. you can add labels in the Logisim). Create output pins for each register and make their values easily observable for testing and validation.

Testing and Verification:

To show that your processor design is working probably, you need to do the following points:

1. Write a set of instructions to check the correctness of ALL instructions in the processor design. Show the correctness of all ALU R-type and I-type instructions. Show the correctness of LW and SW instructions. Also, show the correctness of all branch and jump instructions.
2. Write short codes and loops to check the correctness of a couple of instructions in one program.
3. Write a comparison/conditional based procedure of your choice like the following:
   If (x1>x2) {f=y^2 +2} else {f=y+1}.

Document and record all your test cases and files and include them in the project document.

Project Report:

The report must contain parts highlighting the following points:

I. Cover Page

- Contains the course name, project title, your name (and id) for all team members, and date.

II. Design and Implementation Part

- List the components required to build the chosen design, and the reason why each component is used. Provide drawings of all components and sub-components, and the overall datapath of the processor with full description on the figures. Don't forget to add captions for all figures and call them in the description.
- Provide a detailed description on the control logic and the control signals. Support your description with a **table** giving the control signal values for each instruction. You need to define the control signals and give all possible values.

III. Simulation and Testing Part

- Describe the test cases that you tried to test your processor with the comments describing the cases, their input, and expected output. Don't forget to provide the hex values for all instruction used in the testing cases.
- List all instructions that were tested and work probably. List all instructions that do not work properly.
- Provide snapshots showing test cases and their output results, each snapshot must contain the time and the date of your machine.

IV. Design Alternatives, Issues and Limitations Part

- Provide the drawings of the design alternatives for any component in the design (if any).
- Show the limitations and issues you have faced during the design. You need also to show the not working parts (if any).

V. Teamwork Part

- **Three** students can form a team at max. Write the names of all team members on the project report cover page.
- Team members need to coordinate their work among themselves, so everyone will participate in design, implementation, simulation, and testing.
- Show the work done by each member in the team using a chart (i.e. bar, histogram, pie).

Project Deadlines:

This project consists of three main phases:

A. **Phase One (P1)**: you need to submit only the Logisim circuit file with (**.circ)** extension (named by the students' IDs, e.g. **Part1_ID1_ID2_ID3.circ**), which includes the implementation of the **Register File** and the **ALU** through ITC before end of **Thursday, May 22, 2025**.

B. **Phase Two (P2)**: you need to submit only the Logisim circuit file with (**.circ**) extension (named by the students' IDs, e.g. **Part2_ID1_ID2_ID3.circ**), which includes the implementation of the fully connected datapath unit without the control units (i.e. PC register, Instruction Memory, Register File, ALU, Data Memory) through ITC before end of **Monday, June 2, 2025**.

C. **Phase Three (P3)**: you need to design the control units (i.e. Main control unit and PC control unit) and do the proper connections to finalize the single-cycle processor. At this part, the single-cycle processor project should be finalized before end of **Saturday, June 14, 2025**. You should have enough test cases ready to show that your processor design is fully operational.
If your design is not fully functional then define which instructions/components do not work correctly to avoid losing more points while grading. By this phase, you are required to submit a **.zip** file (named by the students' IDs, e.g. **Part3_ID1_ID2_ID3.zip**) which contains the Logisim circuits for all parts (**.circ**) in a **Folder** named by **Circuits**, the test cases (i.e. codes with hex values as **.txt** files) in a **Folder** named by **Cases**, and the project report (**.pdf**) on **ITC** before the above mentioned deadline.

NOTE: One submission per team is enough and last submission will be only graded for all phases.

Grading Criteria

This is some sort of project competition. So one factor of the evaluation depends on how a project distinguishes itself. The evaluation factors that may distinguish your project:

- The more properly working features the better (i.e. a project of three well designed and implemented features is better than four poorly designed or implemented features).
- How well the parts are integrated with each other to serve the project goal.
- Modular design (Building/using sub-modules for different parts of the project).
- The level of understanding the details of implementation.
- How difficult it is to interface and use the parts (relatively).

The following table summarizes the grading criteria and submission deadlines of all components of the course project.

Project Component	Percentage	Submission Deadline
Phase One	10%	Thursday, May. 22, 2025
Phase Two	15%	Monday, June. 02, 2025
Phase Three & Project Report	50%	Saturday, June. 14, 2025
Discussion & Demo	25%	Sunday 15/6 – Tuesday 17/6

Generally, just by following the guidelines presented in this document, you should get a good score. However, failing to stick to these guidelines may result in a reduction proportional in magnitude to the deviation.

Good luck, *ENCS2380 Instructors*